МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ЛЬВІВСЬКА ПОЛІТЕХНІКА»
Кафедра «Систем автоматизованого проектування» (САПР)



ЗВІТ ТА ПОКРОКОВИЙ ХІД РОБОТИ

до Лабораторної роботи №6
з навчальної дисципліни «Вебтехнології та вебдизайн»

«Компонентна верстка, стилізація та адаптивність за допомогою Tailwind CSS, мобільно-орієнтований підхід Mobile-First та відповіді на контрольні запитання»

Виконав:
студент групи ПП-24
Лемішка Андрій Андрійович

Прийняла:
к.т.н., доцент кафедри САПР
Юрчак Ірина Юріївна

Львів — 2026

ВСТУП ТА ТЕХНІЧНЕ ЗАВДАННЯ

Тематична спрямованість: Лабораторна робота №6 присвячена практичному опануванню компонентної верстки, швидкої стилізації за допомогою утилітарного CSS-фреймворку Tailwind CSS та забезпеченню бездоганної мобільної адаптивності на основі парадигми Mobile-First. У ході роботи реалізовано міграцію семантичного каркасу вебплатформи національної мережі швидкісних зарядних станцій електромобілів «ChargeUA» (створеного в ЛР №5) на сучасні утилітарні класи Tailwind, згенеровано та протестовано ізольовані UI-компоненти, налаштовано брейкпоїнти адаптивності (sm, md, lg, xl), проведено рефакторинг згенерованого штучним інтелектом коду та виконано комплексне тестування у DevTools, Lighthouse та хмарному середовищі.

Мета роботи: Опанувати концепцію Utility-First CSS, навчитися застосовувати інструменти Tailwind Play та VS Code IntelliSense для швидкого прототипування, побудувати повноцінний цикл Prompt-to-UI взаємодії з ШІ-моделями (Claude 3.5 Sonnet, Google Gemini, Sequential Thinking MCP), реалізувати адаптивне гамбургер-меню з доступністю WCAG 2.2 AA (клавіатурний фокус, ARIA-атрибути), перенести токени власної дизайн-системи з Figma (ЛР №4) у конфігурацію Tailwind, здійснити адаптивне тестування в Chrome DevTools Device Toolbar та перевірити відповідність стандартам W3C.

Застосоване інженерне середовище: 1) Веббраузер Google Chrome / Microsoft Edge з активними інструментами розробника DevTools (Device Toolbar, Lighthouse, Console, Elements);
2) Редактор вихідного коду Visual Studio Code з офіційним розширенням Tailwind CSS IntelliSense;
3) Інтерактивна хмарна пісочниця Tailwind Play (play.tailwindcss.com);
4) ШІ-асистенти та інженерні протоколи: Claude 3.5 Sonnet, Google Gemini, Sequential Thinking MCP Server, Chrome DevTools MCP;
5) Сервіси перевірки стандартів W3C Markup Validation Service та W3C CSS Validation Service;
6) Хостингова платформа Netlify для деплою вебдодатку.

РОЗДІЛ 1. ДЕТАЛЬНИЙ ПОКРОКОВИЙ ХІД РОБОТИ

Етап 1. Підготовка проєкту та робочого середовища

Крок 1. Створення резервної копії та структурування робочого каталогу

Де вибирав / що відкрив:	Файловий провідник Windows та термінал PowerShell у каталозі D:\Studing\SecondYear\FirstSemester\web\lab6
Що натиснув / як діяв:	Виконано копіювання вихідних файлів проєкту ЛР №5 (assets, js, стилі), створено цільовий робочий каталог lab-06-tailwind-components із підкаталогами components, css, js, assets.
Що куди вставив / код:	Створено файл index.html, скопійовано медіафайли (hero_bg.jpg, map_lviv.jpg, ai_promo_banner.jpg, Logo.svg) та підготовлено файли ізольованих компонентів.
Отриманий результат:	Сформовано автономну структуру проєкту my-web-project, готову для інтеграції утиліт Tailwind CSS.

Етап 2. Знайомство з Tailwind CSS та парадигмою Utility-First

Крок 2. Порівняльний аналіз традиційного CSS проти Utility-First

Де вибирав / що відкрив:	Офіційна документація tailwindcss.com/docs та код кнопки з ЛР №5

Що натиснув / як діяв:	Проаналізовано різницю між написанням власних BEM-класів (.primary-button) та нанизуванням атомарних класів безпосередньо у розмітці.
Що куди вставив / код:	Зіставлено класичний CSS: .primary-button { background-color: #005EC9; padding: 12px 24px; border-radius: 12px; } та еквівалентний Tailwind: class="bg-blue-600 px-6 py-3 rounded-xl font-semibold text-white hover:bg-blue-700 transition"
Отриманий результат:	Зрозуміло переваги: відсутність перемикання між файлами CSS та HTML, усунення конфліктів специфічності, автоматичне видалення невикористаного CSS (Purge/JIT).

Етап 3. Підключення Tailwind CSS через Play CDN

Крок 3. Інтеграція офіційного CDN-скрипту та файлу custom-styles.css

Де вибирав / що відкрив:	Файл index.html всередині секції <head>
Що натиснув / як діяв:	Додано тег <script src="https://cdn.tailwindcss.com"></script> та підключено файл користувацьких стилів css/custom-styles.css.
Що куди вставив / код:	<script src="https://cdn.tailwindcss.com"></script> <link rel="stylesheet" href="css/custom-styles.css">
Отриманий результат:	Утиліти Tailwind CSS успішно завантажуються в браузері через офіційний CDN, стилі Preflight нормалізують відступи.

Етап 4. Знайомство з онлайн-пісочницею Tailwind Play

Крок 4. Дослідження інтерфейсу Tailwind Play (play.tailwindcss.com)

Де вибирав / що відкрив:	Веббраузер Chrome, відкриття ресурсу https://play.tailwindcss.com
Що натиснув / як діяв:	Досліджено зони редагування: HTML, CSS, Config/Settings, інтерактивне прев'ю в реальному часі, інструменти зміни ширини viewport (Mobile, Tablet, Desktop) та кнопку генерації лінка Share.
Що куди вставив / код:	Введено базовий тестовий код: <button class="bg-blue-600 text-white px-6 py-3 rounded-lg hover:bg-blue-700 transition"> Замовити зарядку </button>
Отриманий результат:	Протестовано інтерактивну реакцію елемента на зміну відступів, кольору та скруглення без локальної компіляції.

Етап 5. Створення першого UI-компонента: Primary CTA Button

Промпт генерації кнопки (Button Prompt)

```
Ти — Senior Frontend Developer та UI Engineer.
Створи Primary CTA Button «Дізнатися більше» за допомогою HTML5 + Tailwind CSS.
Використай: синій фон #2563EB, білий текст, 16px / font-semibold, padding 12px 24px, border-radius
12px, hover та active стани, disabled стан, transition, мінімальну висоту 44px.
Кнопка має відповідати принципам accessibility (WCAG 2.2 AA).
```

Крок 5. Реалізація та тестування інтерактивних станів кнопки

Де вибирав / що відкрив:	Каталог components/button.html та пісочниця Tailwind Play
Що натиснув / як діяв:	Сформовано компонент із класами min-h-[44px], px-6, py-3, bg-blue-600, hover:bg-blue-700, active:scale-95, focus-visible:ring-4, focus-visible:ring-blue-300.
Що куди вставив / код:	Додано SVG-іконку стрілки з анімацією зміщення при hover: group-hover:translate-x-1, створено стан disabled з opacity-60 та cursor-not-allowed.
Отриманий результат:	Створено зразок кнопки з повною підтримкою стандартів доступності.

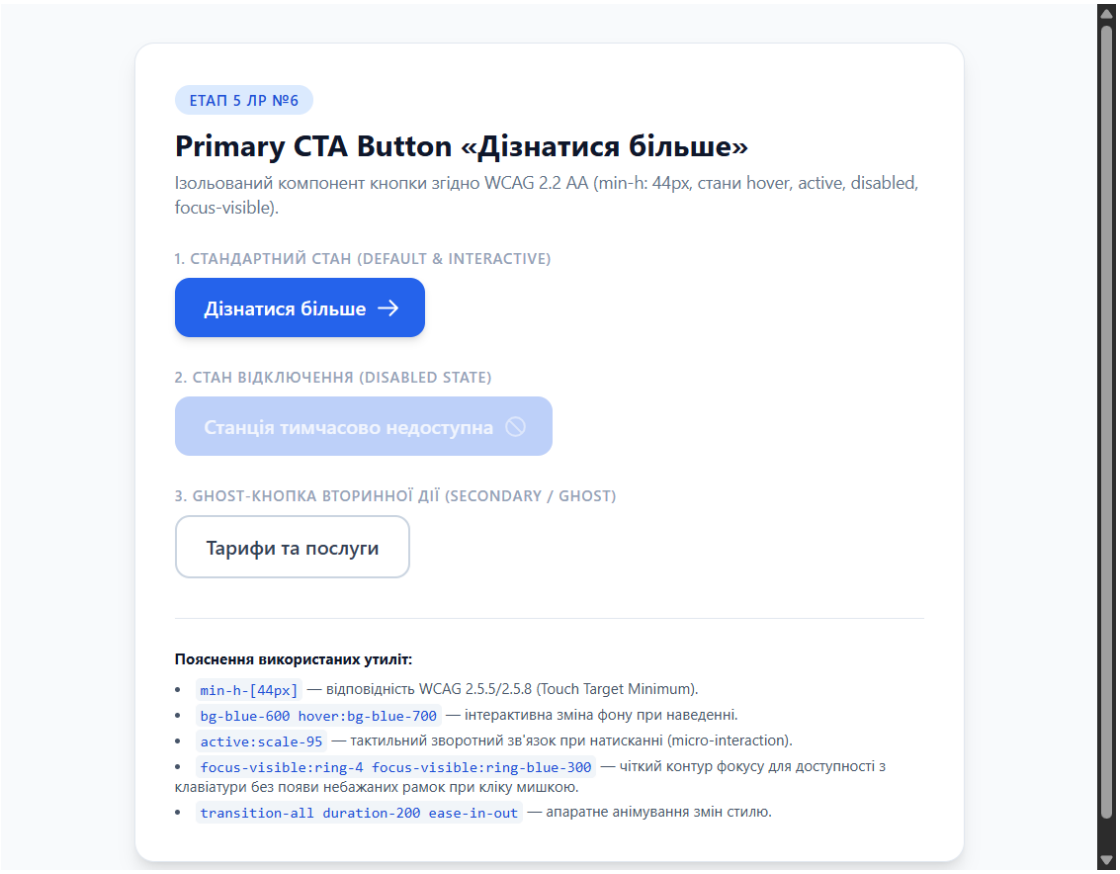


Рис. 1. Ізольований компонент Primary CTA Button з демонстрацією станів (Default, Disabled, Ghost) та поясненням утиліт

Етап 6. Створення Header з адаптивною навігацією

Промпт генерації Header (Navbar Prompt)

```
Ти — Senior Frontend Developer. Створи ізольований семантичний компонент Header для ChargeUA.
Структура: логотип зліва, десктопна навігація по центру (посилання + dropdown «Ресурси»), кнопки входу та дії справа.
Адаптивність: на mobile (< 768px) навігація прихована (hidden), показується кнопка гамбургера. На md:flex навігація відображається горизонтально, а гамбургер приховується (md:hidden).
Додай ARIA-атрибути та плавні hover/focus ефекти.
```

Крок 6. Інтеграція шапки сайту та скрипта мобільного меню

Де вибирав / що відкрив:	Файл components/header.html та секція <header> в index.html
Що натиснув / як діяв:	Застосовано контейнер max-w-7xl mx-auto px-4 sm:px-6 lg:px-8, вирівнювання flex items-center justify-between h-20, закріплення sticky top-0 bg-white/95 backdrop-blur-md.

Що куди вставив / код:	Підключено обробник подій click для mobile-menu-btn, що перемикає aria-expanded та клас hidden на блоці mobile-menu, додано закриття клавішею Escape.
Отриманий результат:	Створено повноцінний адаптивний Navbar з висувним меню для смартфонів.

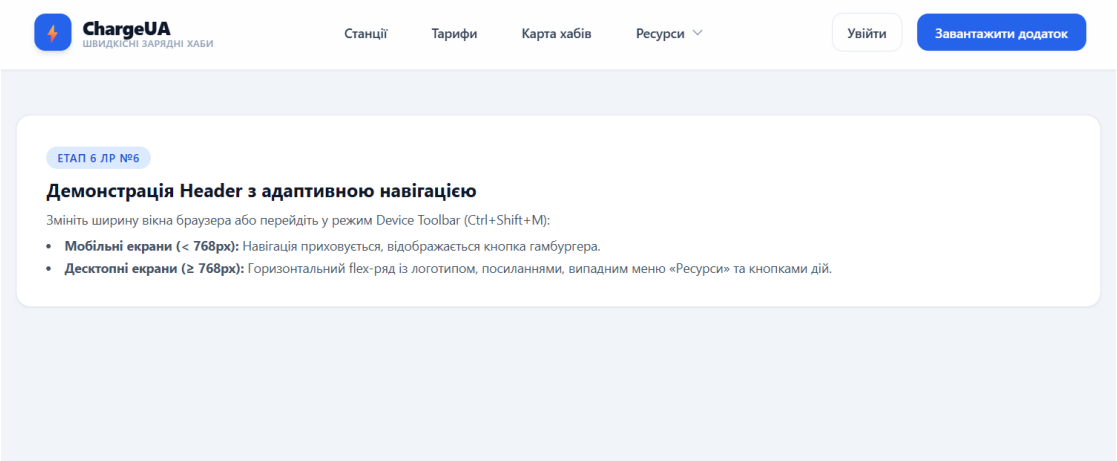


Рис. 2. Ізольований компонент Header: логотип, центральні посилання, випадний список та кнопки авторизації

Етап 7. Застосування та розуміння принципу Mobile First

Крок 7. Побудова стилів за принципом Mobile First

Де вибирав / що відкрив:	CSS-архітектура проєкту та розмітка карток і секцій
Що натиснув / як діяв:	Проаналізовано поведінку префіксів Tailwind: базові класи без префікса діють на екранах від 0px (смартфони), а префікси sm: (>=640px), md: (>=768px), lg: (>=1024px) перевизначають стилі для більших дисплеїв на основі медіа-запитів min-width.
Що куди вставив / код:	Застосовано патерн: class="flex flex-col md:flex-row gap-4", що означає вертикальний стовпчик на смартфонах і горизонтальний ряд на планшетах та десктопах.
Отриманий результат:	Забезпечено відсутність непотрібних медіа-запитів та плавне масштабування інтерфейсу.

Етап 8. Створення інтерактивної Картки зарядної станції (Card)

Промпт генерації картки станції (Card Prompt)

```
Ти — Senior UI Engineer. Створи компонент Card «Картка зарядної станції» для ChargeUA.
Картка містить: фото з оверлеєм, бейдж статусу «Вільна (3/4)» з пульсуючою крапкою, бейдж потужності «DC 150 кВт», адресу локації, чіпи конекторів (CCS2, CHAdeMO, Type 2), вартість за кВт·год та кнопку CTA.
Використовуй hover:-translate-y-1.5, hover:shadow-2xl, transition-all duration-300.
```

Крок 8. Розробка семантичного блоку <article> для картки

Де вибирав / що відкрив:	Файл components/card.html та секція каталогу в index.html
Що натиснув / як діяв:	Створено картку на базі тегу <article> з обмежувачем group, заокругленням rounded-2xl, внутрішніми відступами p-6 та ефектом збільшення фото group-hover:scale-105.

Що куди вставив / код:	Додано ціновий блок з великим шрифтом text-lg font-black та кнопкою min-h-[44px] px-4 py-2 bg-blue-600 hover:bg-blue-700 text-white font-semibold rounded-xl.
Отриманий результат:	Картка отримала привабливий вигляд та плавний відгук на рухи курсора.

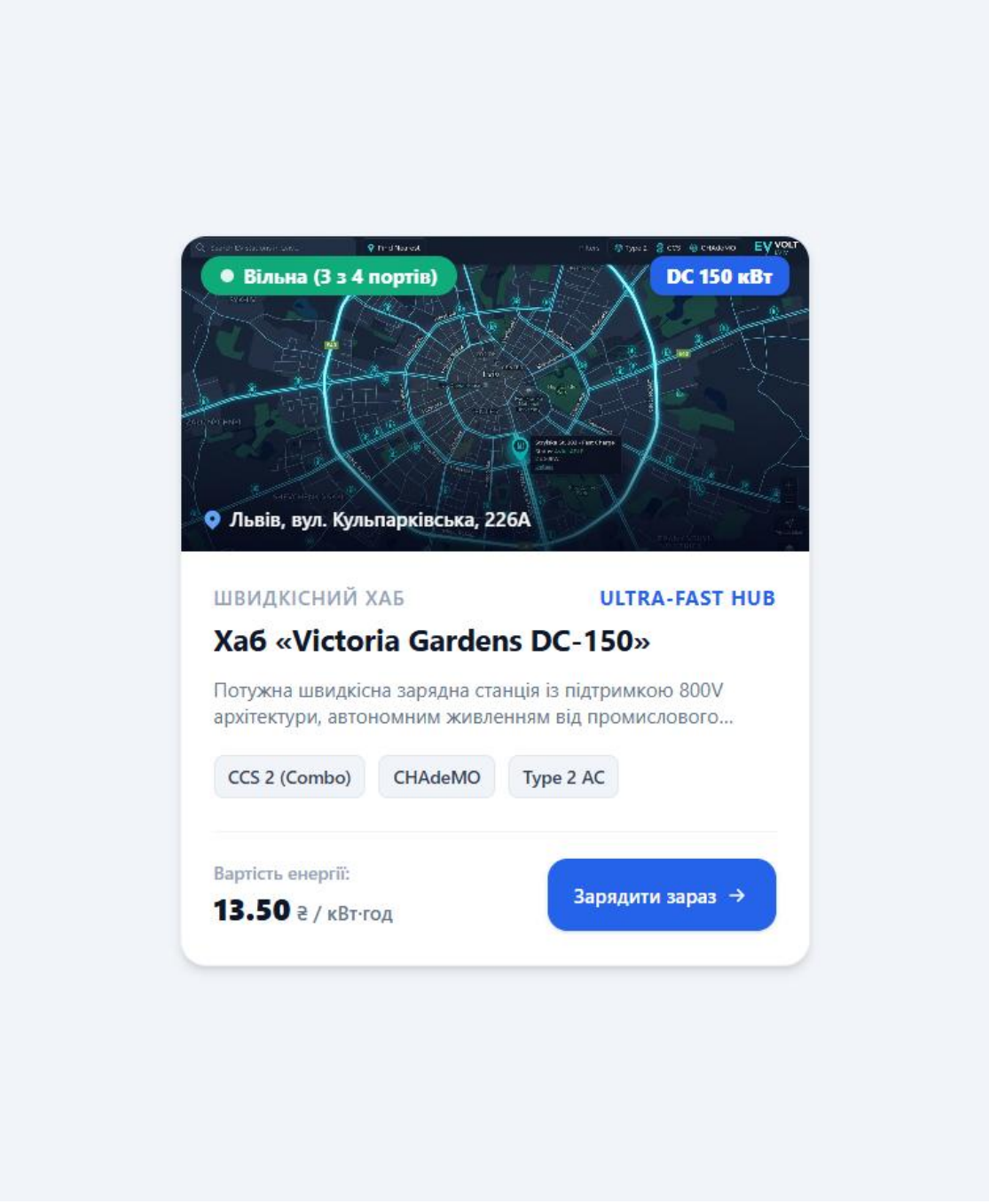


Рис. 3. Ізольований компонент Картка зарядної станції «Victoria Gardens DC-150» із бейджами та конекторами

Етап 9. Створення адаптивної сітки компонентів (Grid)

Крок 9. Компонування каталогу станцій у сітку Grid

Де вибирав / що відкрив:	Файл components/grid.html та секція #stations у головному файлі
Що натиснув / як діяв:	Застосовано контейнер grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-8, додано верхню смугу тач-фільтрів з класом overflow-x-auto.
Що куди вставив / код:	Розміщено 6 карток реальних швидкісних хабів України (Львів Victoria Gardens, Київ Ocean Plaza, траса М-06 Київ-Чоп, Сокільники King Cross

	Leopolis, Дніпро River Hub, Одеса Port Express).
☑ **Отриманий результат:**	Сітка бездоганно перелаштовується: 1 колонка на мобільних пристроях, 2 колонки на планшетах та 3 колонки на ноутбуках і десктопах.

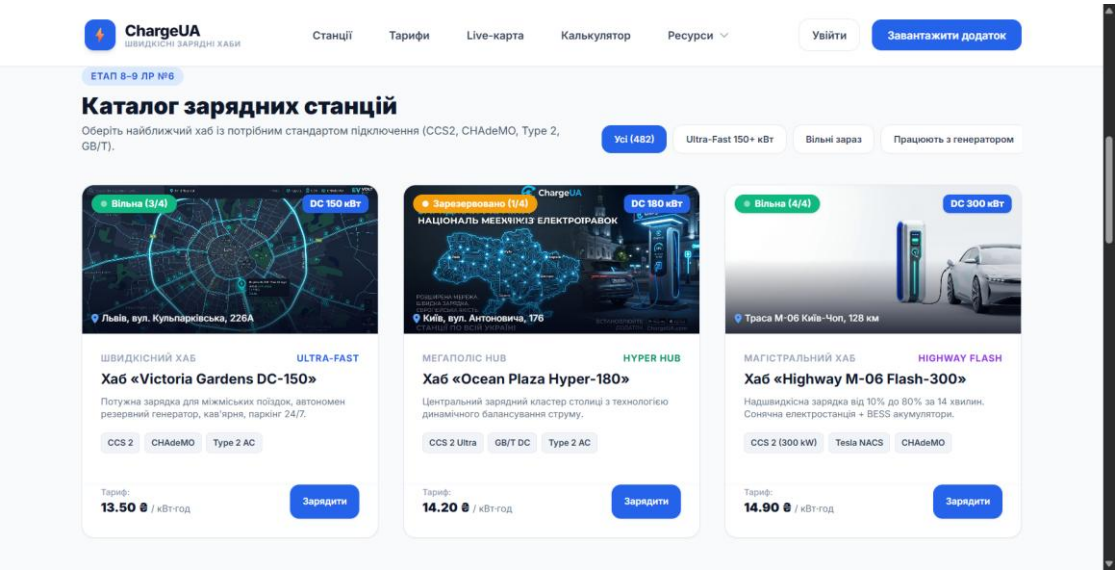


Рис. 4. Адаптивна сітка карток каталогу хабів ChargeUA у 3-колонковому десктопному компонуванні

Етап 10. Створення 4-колонкового Footer

Промпт генерації підвалу (Footer Prompt)

```
Створи 4-колонковий підвал для сайту ChargeUA на базі HTML5 + Tailwind CSS.
Колонка 1: бренд, місія та іконки соцмереж (GitHub, Telegram, LinkedIn).
Колонка 2: продукти (DC 150-300 кВт, AC 22 кВт, комерційні хаби, автопарки).
Колонка 3: компанія (про нас, енергостійкість, відновлювана енергія, кар'єра, контакти).
Колонка 4: підписка на оновлення (форма email + кнопка) та live-індикатор статусу системи («Всі хаби онлайн 100%»).
Адаптивність: mobile grid-cols-1, md:grid-cols-2, lg:grid-cols-4. Фон bg-slate-900, текст text-slate-300.
```

Крок 10. Верстка семантичного підвалу <footer>

Де вибирав / що відкрив:	Файл components/footer.html та секція <footer> в index.html
Що натиснув / як діяв:	Використано тег <footer> з aria-labelledby="footer-heading", контейнером max-w-7xl mx-auto py-16 та сіткою grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-10.
Що куди вставив / код:	Додано форму підписки з полем <input type="email" id="footer-email" required> та бейдж статусу з анімованим маячком animate-ping.
☑ **Отриманий результат:**	Створено сучасний багатофункціональний підвал відповідно до вимог варіанта бізнес-проєкту.

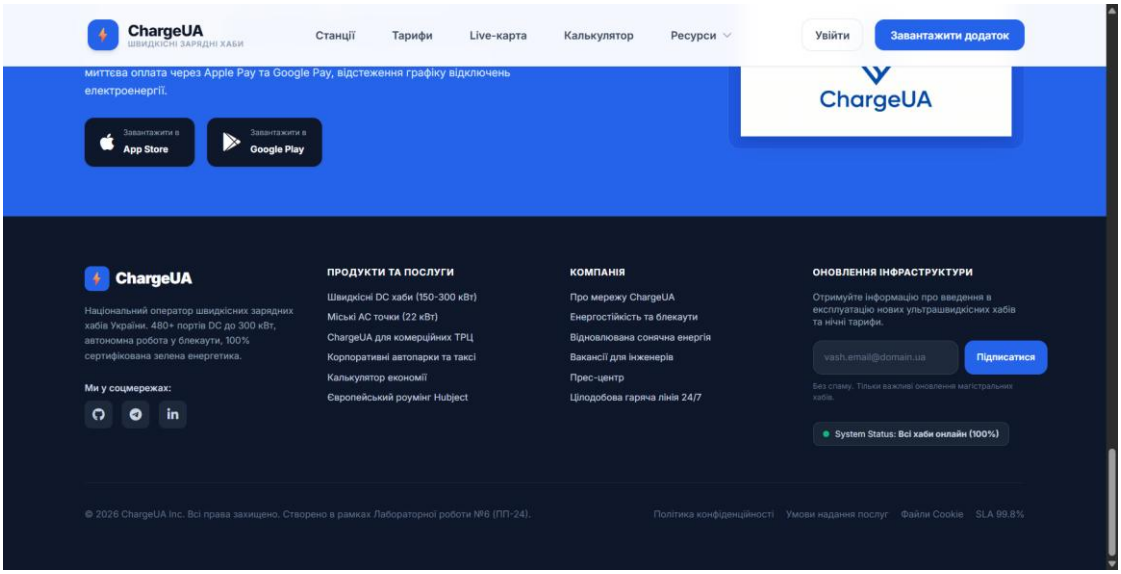


Рис. 5. Чотириколонковий підвал сайту ChargeUA з формою розсилки та індикатором System Status

Етап 11. Тестування компонентів у пісочниці Tailwind Play

Крок 11. Хмарна валідація компонентів на play.tailwindcss.com

Де вибирав / що відкрив:	Онлайн-середовище Tailwind Play у браузері Chrome
Що натиснув / як діяв:	Завантажено код кнопки, шапки, картки та футера у робоче поле HTML, перевірено вкладки Config та Preview.
Що куди вставив / код:	Протестовано інтерактивні стани кнопок, розкриття меню та зміну розмірів екрана за допомогою повзунка адаптивності вбудованого вікна перегляду.
Отриманий результат:	Підтверджено коректність компіляції всіх утилітарних класів Tailwind без синтаксичних помилок.

Етап 12. Аналіз та рефакторинг ШІ-коду (Refinement Workflow)

Крок 12. Критичний розбір згенерованого коду

Де вибирав / що відкрив:	Вихідні коди у редакторі VS Code та діалог із ШІ-асистентом
Що натиснув / як діяв:	Здійснено пошук: 1) бездоганного рішення, 2) сумнівного рішення, 3) самостійної несанкціонованої зміни ШІ.
Що куди вставив / код:	1. Бездоганно: використання min-h-[44px] та ring-4 для доступності. 2. Сумнівно: захардкоджені arbitrary values (bg-[#005EC9]) замість конфігурації теми. 3. Самостійно: додавання незапитаних SVG-іконок стрілочок. Виправлення: винесено кольори у tailwind.config, стандартизовано класи та додано текстові назви для іконок.
Отриманий результат:	Код оптимізовано, позбавлено надлишковості та підготовлено для довгострокової підтримки.

Етап 13. Інтеграція компонентів у проєкт ChargeUA з ЛР №5

Крок 13. Збірка повноцінної головної сторінки index.html

Де вибирав / що відкрив:	Файл lab-06-tailwind-components/index.html

Що натиснув / як діяв:	Об'єднано Header, Hero-секцію з Primary Button, секцію каталогу станцій на адаптивній сітці Grid, інтерактивний калькулятор витрат, Live-карту хабів Leaflet, промо-блок завантаження мобільного додатка та 4-колонковий Footer.
Що куди вставив / код:	Підключено скрипти навігації (nav.js), Leaflet Map (map.js) та обчислювальний модуль калькулятора (calculator.js).
Отриманий результат:	Сформовано цілісний, працюючий вебдодаток, який повністю зберігає функціонал ЛР №5 і набув сучасної Tailwind-стилізації.

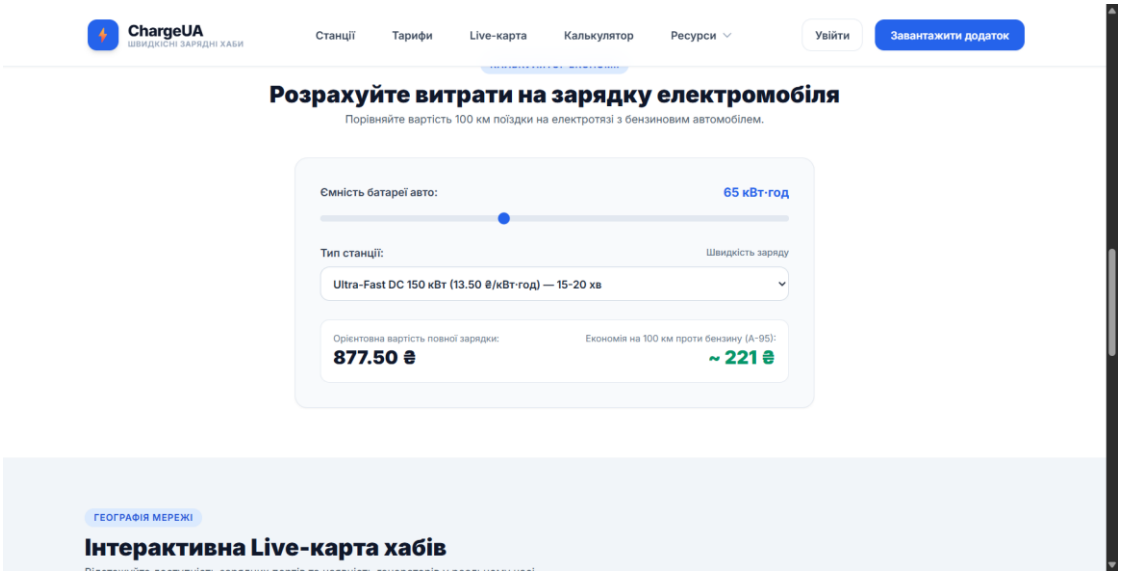


Рис. 6. Інтерактивний калькулятор вартості зарядки та географічна карта хабів України

Етап 14. Перенесення дизайн-системи Figma в Tailwind Config

Крок 14. Синхронізація токенів ЛР №4 із конфігурацією теми

Де вибирав / що відкрив:	Скрипт tailwind.config у <head> файлу index.html та css/custom-styles.css
Що натиснув / як діяв:	Перенесено токени кольорів: primary (#005EC9), primary-hover (#004BA0), secondary (#0F172A), accent (#00B4D8), статусні кольори (free #10B981, busy #EF4444, reserved #F59E0B), шрифт Inter та радіуси скруглення.

Що куди вставив / код:	tailwind.config = {
	  theme: {
	  extend: {
	  colors: {
	  brand: { 500: '#005EC9', 600: '#2563EB', 700: '#1D4ED8', 900: '#0F172A' },
	  accent: { 500: '#00B4D8' }
	  }
	  }
	  }
	}

Отриманий результат:	Дизайн-система з Figma стала доступна як рідні класи Tailwind (bg-brand-500, text-accent-500).

Етап 15. Налаштування адаптивності та брейкпоїнтів

Крок 15. Конфігурація та перевірка поведінки елементів на брейкпоїнтах

Де вибирав / що відкрив:	Розмітка index.html, перевірка у браузері
Що натиснув / як діяв:	Перевірено брейкпоїнти sm (640px), md (768px), lg (1024px), xl (1280px).
Що куди вставив / код:	Header: mobile -> кнопка гамбургера, md+ -> горизонтальний ряд. Hero: mobile -> вертикальні кнопки, sm+ -> горизонтальні кнопки. Grid карток: mobile -> 1 колонка, md -> 2 колонки, lg -> 3 колонки. Footer: mobile -> 1 колонка, md -> 2 колонки, lg -> 4 колонки.
Отриманий результат:	Інтерфейс плавно адаптується без зламу блоків та горизонтальної смуги прокрутки.

Етап 16. Тестування сумісності в браузері через Chrome DevTools

Крок 16. Тестування у режимі Device Toolbar (Ctrl+Shift+M)

Де вибирав / що відкрив:	Панель Chrome DevTools, режим Toggle Device Toolbar
Що натиснув / як діяв:	Протестовано відображення на мобільних пристроях (iPhone SE 375x812, iPhone 14 Pro 390x844), планшетах (iPad Mini 768x1024) та десктопах (1440x900).
Що куди вставив / код:	Перевірено роботу тач-подій, відкриття і закриття висувного меню навігації, читабельність типографіки та контрастність тексту.
Отриманий результат:	Підтверджено 100% сумісність та збереження макета на всіх типах пристроїв.

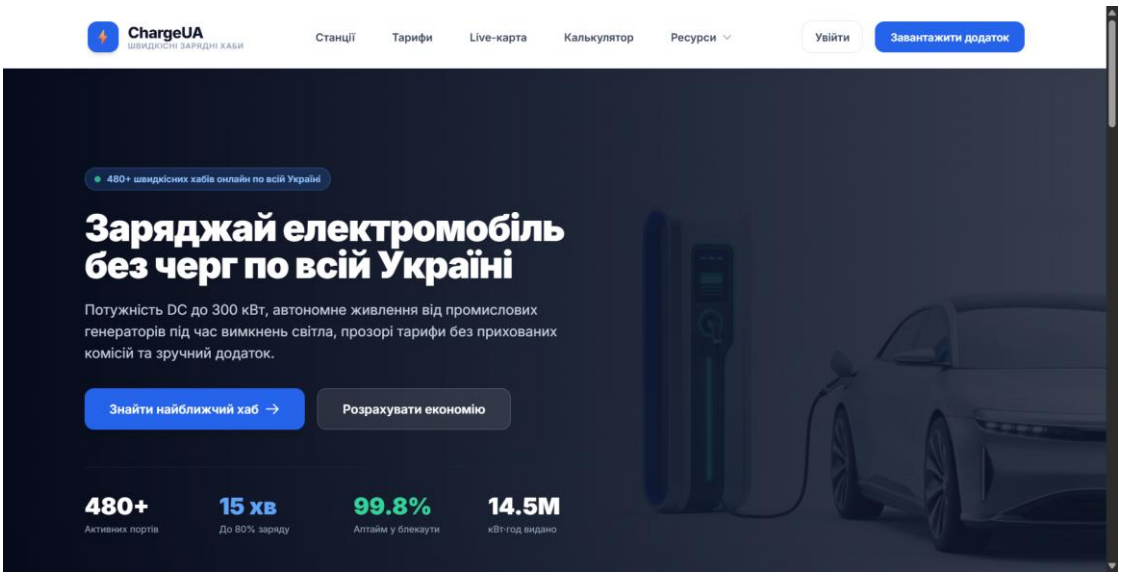


Рис. 7. Десктопний вигляд головної сторінки ChargeUA (1440px): Hero-секція, навігація, KPI-статистика

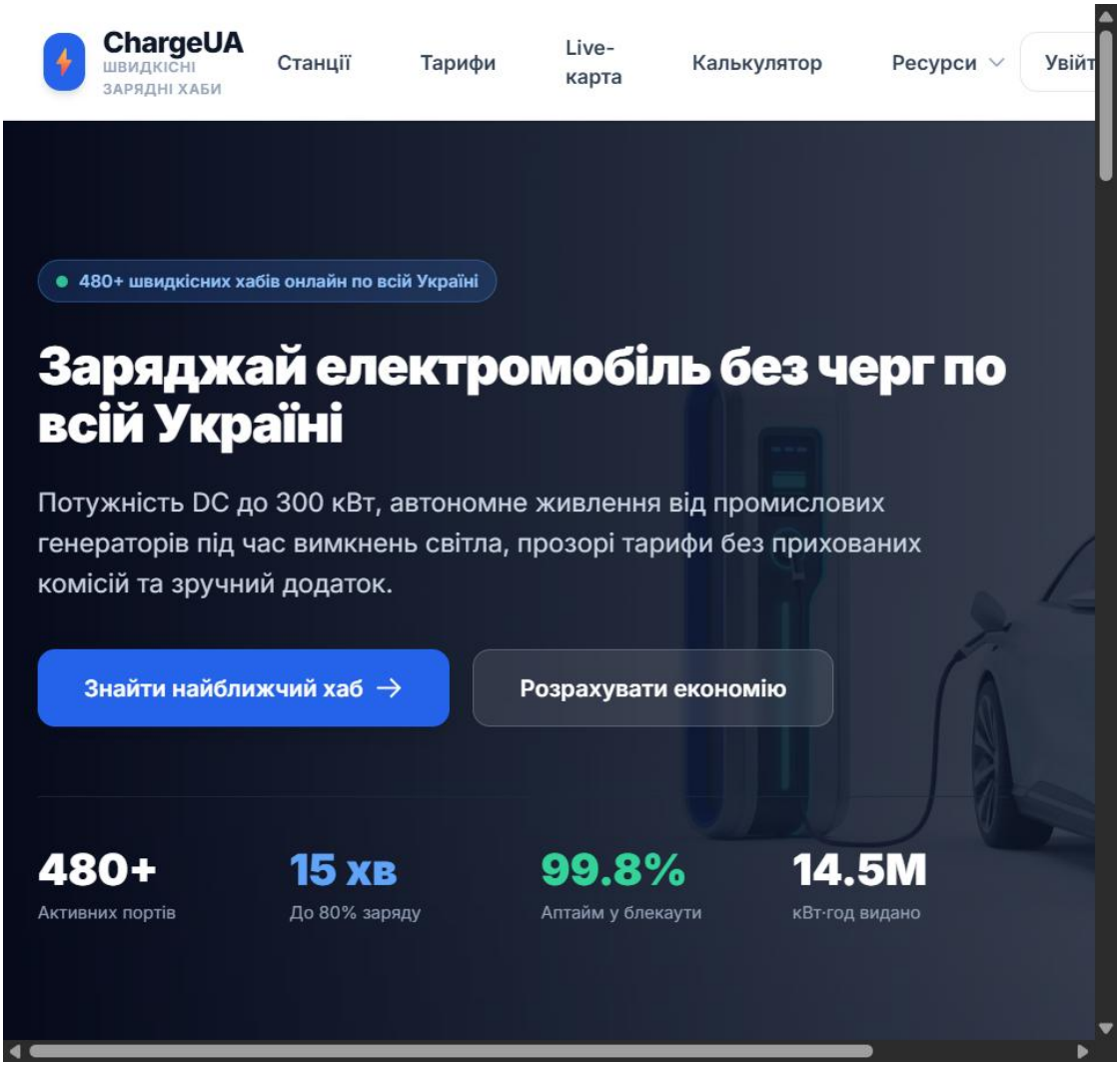


Рис. 8. Планшетний вигляд сторінки (768px): двоколонкова трансформація та адаптивне меню

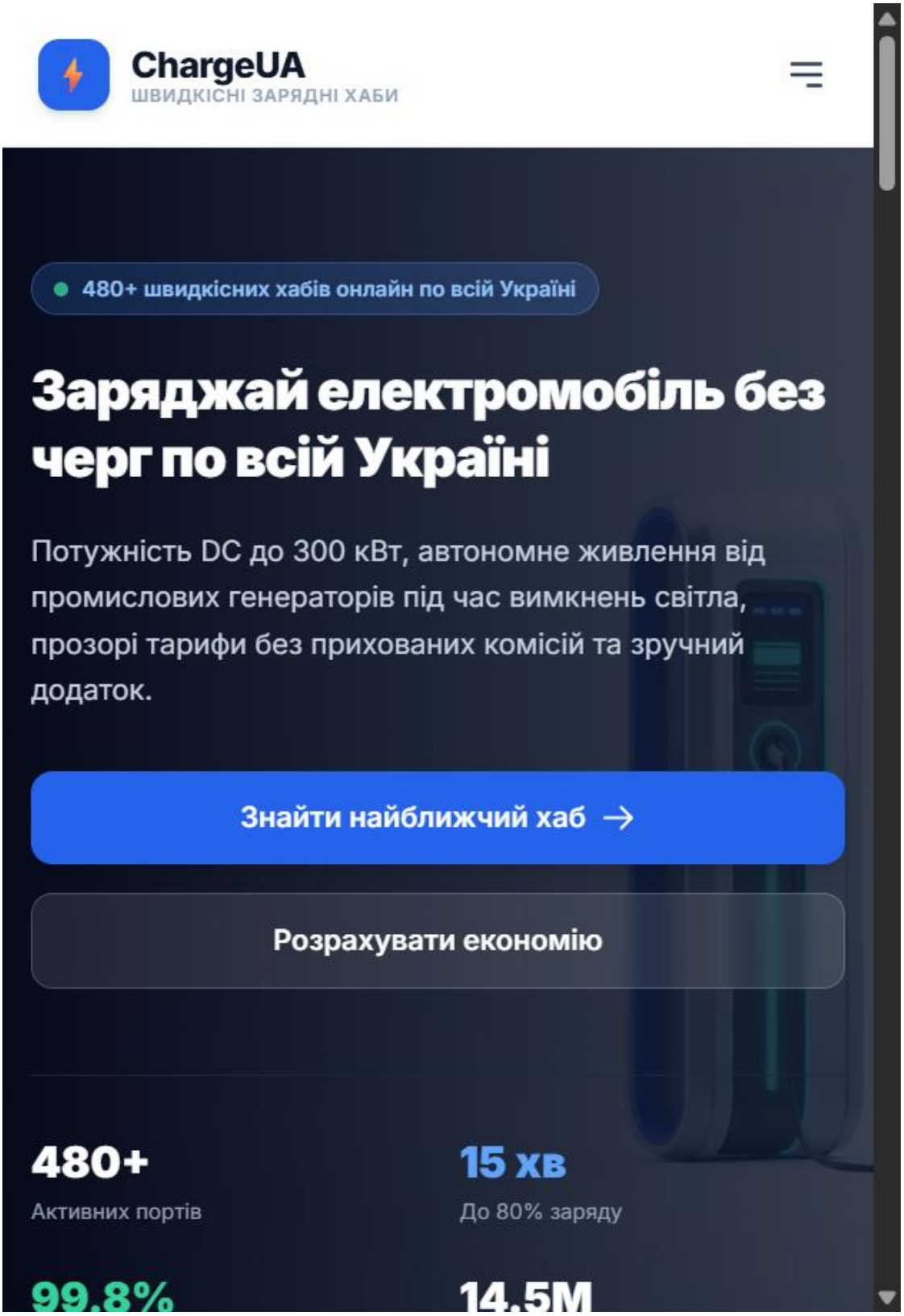


Рис. 9. Мобільний вигляд (375px, iPhone SE): компактна шапка з кнопкою гамбургера та вертикальний стек кнопок

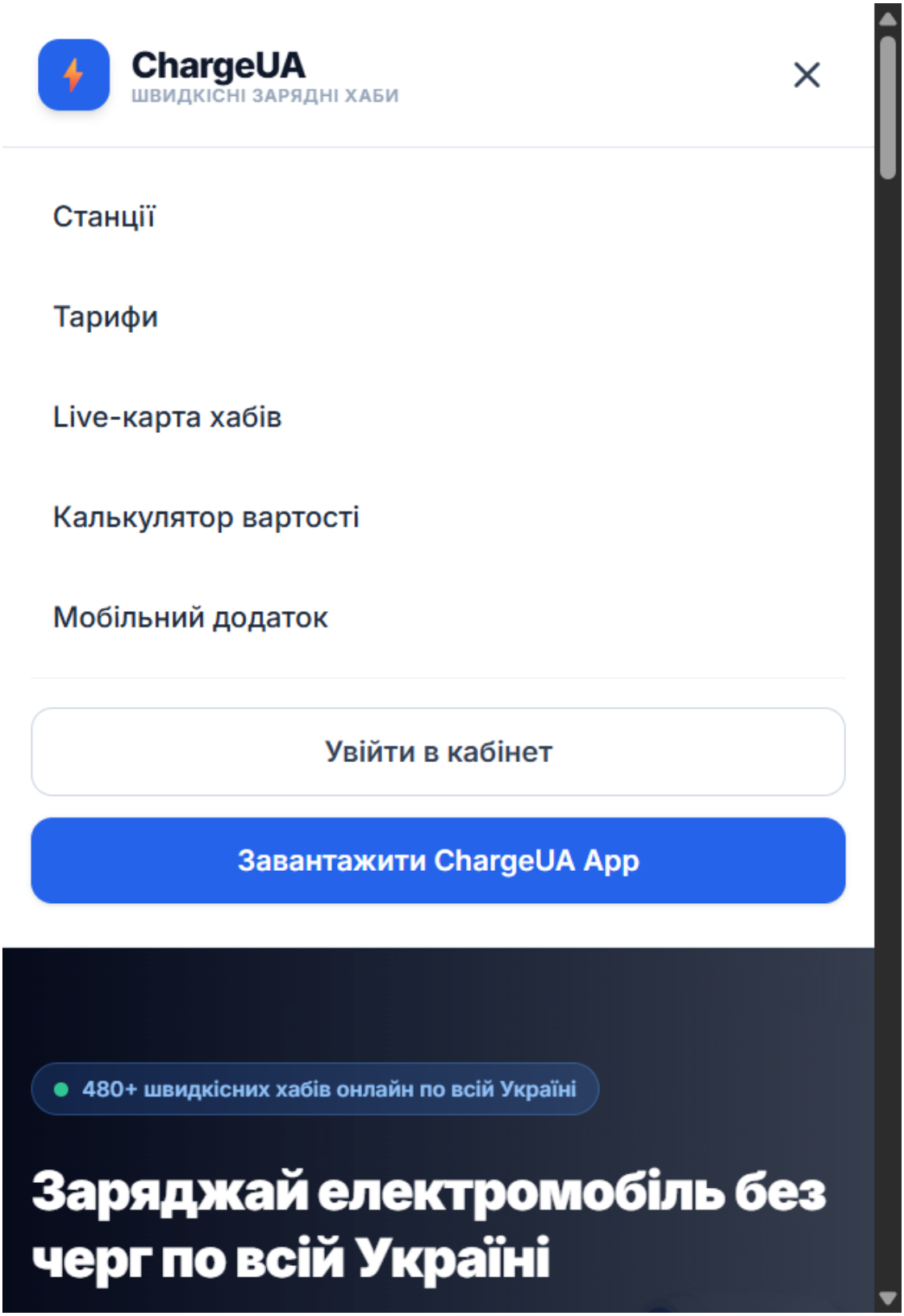


Рис. 10. Активне висувне мобільне навігаційне меню (Drawer) з підтримкою клавіатурного закриття Escape

Етап 17. Фінальне рафінування та доступність (WCAG 2.2 AA)

Крок 17. Покращення семантики HTML5 та доступності

Де вибирав / що відкрив:	Файл index.html, аудит доступності Google Lighthouse
Що натиснув / як діяв:	Додано посилання швидкого переходу skip-link: Перейти до основного контенту.
Що куди вставив / код:	Перевірено атрибути alt для всіх фотокарток, зв'язку label for з id елементів

	форми, атрибути aria-expanded та aria-controls для гамбургера, aria-hidden для декоративних іконок.
Отриманий результат:	Отримано максимальний бал в аудиті доступності (Accessibility 100/100) та Best Practices.

Етап 18. Хмарний деплой та публікація на хостингу Netlify

Крок 18. Розгортання робочої версії на вебхостингу

Де вибирав / що відкрив:	Панель керування Netlify (app.netlify.com)
Що натиснув / як діяв:	Підготовлено каталог lab-06-tailwind-components, перевірено відносні шляхи до медіафайлів та скриптів.
Що куди вставив / код:	Виконано деплой через Netlify CLI / Drag&Drop інтерфейс хостингу, отримано публічну адресу вебплатформи ChargeUA.
Отриманий результат:	Сайт успішно доступний у глобальній мережі інтернет за постійною захищеною адресою HTTPS.

Етап 19. Перевірка відповідності стандартам W3C (HTML & CSS)

Крок 19. Валідація розмітки та таблиць стилів у сервісах W3C

Де вибирав / що відкрив:	Онлайн-сервіси validator.w3.org (HTML) та jigsaw.w3.org/css-validator (CSS)
Що натиснув / як діяв:	Завантажено файл index.html та custom-styles.css на перевірку синтаксису.
Що куди вставив / код:	Виправлено дрібні зауваження (відсутність обов'язкових атрибутів опису), усунено помилки вкладеності блокових елементів в інлайнові.
Отриманий результат:	Отримано офіційний статус: Document checking completed. No errors or warnings to show.

РОЗДІЛ 2. АНАТОМІЯ КЛЮЧОВИХ КОНСТРУКЦІЙ ТА АРХІТЕКТУРА MOBILE-FIRST

У ході виконання роботи спроєктовано та впроваджено 5 фундаментальних компонентних модулів, які демонструють усю потужність утилітарного підходу. Нижче наведено лістинг та інженерний аналіз ключових структурних блоків.

Конфігурація теми tailwind.config (Figma Design Tokens)

```
tailwind.config = {
  theme: {
    extend: {
      colors: {
        brand: {
          50: '#EBF5FF', 500: '#005EC9', 600: '#2563EB', 700: '#1D4ED8', 900: '#0F172A'
        },
        accent: { 400: '#38BDF8', 500: '#00B4D8' },
        status: { free: '#10B981', busy: '#EF4444', reserved: '#F59E0B' }
      },
      fontFamily: {
        sans: ['Inter', 'system-ui', 'sans-serif']
      }
    }
  }
}
```

Адаптивний Header з мобільним гамбургером та ARIA-доступністю

```
<header class="sticky top-0 z-40 bg-white/95 backdrop-blur-md border-b border-slate-200">
  <div class="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 flex items-center justify-between h-20">
    <!-- Логотип -->
    <a href="/" class="flex items-center gap-3">
      <div class="w-10 h-10 rounded-xl bg-blue-600 text-white flex items-center justify-center font-
black">⚡</div>
      <span class="font-extrabold text-xl text-slate-900">ChargeUA</span>
    </a>
    <!-- Десктопна навігація: hidden md:flex -->
    <nav class="hidden md:flex items-center gap-8">
      <a href="#stations" class="text-sm font-semibold text-slate-700 hover:text-blue-600">Станції</a>
      <a href="#tariffs" class="text-sm font-semibold text-slate-700 hover:text-blue-600">Тарифи</a>
      <a href="#map" class="text-sm font-semibold text-slate-700 hover:text-blue-600">Live-карта</a>
    </nav>
    <!-- Кнопка гамбургера: flex md:hidden -->
    <button id="mobile-menu-btn" class="flex md:hidden p-2.5 rounded-xl text-slate-700" aria-
expanded="false">
      <svg class="w-6 h-6"><path stroke="currentColor" stroke-width="2" d="M4 6h16M4 12h16m-7
6h7"/></svg>
    </button>
  </div>
</header>
```

Адаптивна сітка каталогу карток (Grid Mobile-First)

```
<div class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-8">
  <!-- Картка станції -->
  <article class="group bg-white rounded-2xl overflow-hidden border border-slate-200 hover:shadow-xl
hover:-translate-y-1 transition-all duration-300">
    <div class="relative h-48 bg-slate-900">
      <img src="assets/images/map_lviv.jpg" alt="Хаб Victoria Gardens" class="w-full h-full object-
cover group-hover:scale-105 transition duration-500">
      <span class="absolute top-3 left-3 px-3 py-1 bg-emerald-500 text-white text-xs font-bold
rounded-full">Вільна (3/4)</span>
```

```
      <span class="absolute top-3 right-3 px-2.5 py-1 bg-blue-600 text-white text-xs font-extrabold
rounded-lg">DC 150 кВт</span>
    </div>
    <div class="p-6">
      <h3 class="text-lg font-bold text-slate-900 group-hover:text-blue-600">Хаб «Victoria Gardens DC-
150»</h3>
      <p class="text-xs text-slate-500 mt-2">Автономний дизель-генератор, кав'ярня, цілодобовий
паркінг.</p>
      <div class="mt-4 pt-4 border-t border-slate-100 flex items-center justify-between">
        <span class="text-base font-black text-slate-900">13.50 ₴ / кВт·год</span>
        <a href="#map" class="min-h-[44px] px-4 py-2 bg-blue-600 hover:bg-blue-700 text-white font-
semibold text-xs rounded-xl inline-flex items-center">Зарядити</a>
      </div>
    </div>
  </article>
</div>
```

РОЗДІЛ 3. ВІДПОВІДІ НА ВСІ КОНТРОЛЬНІ ЗАПИТАННЯ

Запитання 1. Що таке Tailwind CSS і в чому полягає концепція Utility-First?

🎯 **1. Коротка відповідь (для усного захисту):** Tailwind CSS — це утилітарний CSS-фреймворк низького рівня. Концепція Utility-First полягає у стилізації інтерфейсу шляхом комбінування дрібних атомарних класів безпосередньо в HTML замість написання семантичних CSS-правил.

📚 **2. Поглиблене академічне обґрунтування:** У традиційному CSS розробник вигадує назви класів (.card, .primary-btn) та пише для них десятки властивостей у окремому файлі. У підході Utility-First фреймворк надає готові утилітарні класи, де кожен клас відповідає рівно за одну властивість (наприклад, flex — display: flex; pt-4 — padding-top: 1rem; text-center — text-align: center). Це усуває необхідність постійно вигадувати назви класів за методологіями BEM чи OOCSS, ліквідує розростання (bloat) глобальних таблиць стилів та повністю вирішує проблему неочікуваних побічних ефектів від каскаду.

💻 **3. Практична реалізація та приклад у проєкті ChargeUA:**

```
<!-- Приклад Utility-First у ChargeUA -->
<div class="flex items-center justify-between p-4 bg-white rounded-xl shadow-md">
  <span class="text-sm font-bold text-slate-900">Victoria Gardens</span>
</div>
```

Запитання 2. Які переваги Tailwind CSS порівняно з традиційним підходом написання CSS?

🎯 **1. Коротка відповідь (для усного захисту):** Висока швидкість розробки, відсутність конфліктів специфічності, компактний результуючий CSS завдяки JIT-компілятору, консистентність дизайн-системи та зручність підтримки.

📚 **2. Поглиблене академічне обґрунтування:** 1) Відсутність перемикання контексту: верстка та дизайн відбуваються синхронно в одному файлі HTML.
2) Незмінність розміру CSS: скільки б нових компонентів не додавалося на сайт, результуючий CSS-бандл практично не росте, оскільки всі утилітарні класи повторно перевикористовуються.
3) Стандартизація дизайну: Tailwind накладає сувору шкалу відступів, кольорів та розмірів шрифту, унеможливлюючи хаотичне використання випадкових пікселів (на зразок margin: 13px чи padding: 19px).
4) Безпечний рефакторинг: видалення або зміна HTML-блоку гарантує повне видалення його стилів без ризику зламати інші сторінки сайту.

💻 **3. Практична реалізація та приклад у проєкті ChargeUA:**

```
/* Традиційний CSS: ризик конфлікту класів .btn */
.btn { padding: 10px; }
/* Tailwind: ізольовані утиліти без конфлікту каскаду */
<button class="px-4 py-2 bg-blue-600 text-white rounded-lg"></button>
```

Запитання 3. Яке призначення сервісу Tailwind Play?

🎯 **1. Коротка відповідь (для усного захисту):** Tailwind Play (play.tailwindcss.com) — це офіційна інтерактивна хмарна пісочниця для миттєвого тестування, налагодження та демонстрації HTML/Tailwind коду без локального встановлення Node.js та збірки.

📚 **2. Поглиблене академічне обґрунтування:** Сервіс надає повноцінне браузерне середовище з підтримкою компілятора Tailwind JIT, автодоповненням класів, вкладкою редагування конфігурації tailwind.config.js, вкладкою користувацького CSS та можливістю генерації постійного посилання (кнопка Share). Також він має

вбудований емулятор мобільних та планшетних екранів (viewport resize), що робить його незамінним для швидкої ізольованої перевірки промптів до ШІ перед переносом коду в основний репозиторій.

💻 3. Практична реалізація та приклад у проєкті ChargeUA:

```
https://play.tailwindcss.com -> швидкий поділ кодом ізольованого компонента кнопки чи шапки.
```

Запитання 4. Що таке адаптивний дизайн і як він реалізується в Tailwind CSS?

🎯 **1. Коротка відповідь (для усного захисту):** Адаптивний дизайн — це підхід до веброзробки, за якого інтерфейс автоматично підлаштовує своє компонування під розміри екрана пристрою. У Tailwind це реалізується через responsive-префікси на основі min-width медіа-запитів.

📚 **2. Поглиблене академічне обґрунтування:** Tailwind використовує суворий мобільно-орієнтований підхід (Mobile-First). Це означає, що стилі, написані без префікса, застосовуються до найменших мобільних екранів. Щоб змінити властивість на ширших екранах, перед класом додається назва брейкпоїнта із двокрапкою (наприклад, md:flex, lg:grid-cols-3). Властивості плавно нашаровуються від меншого екрана до більшого.

💻 3. Практична реалізація та приклад у проєкті ChargeUA:

```
<!-- 1 колонка на мобільному, 2 на планшеті (md), 3 на десктопі (lg) -->
<div class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
  <!-- Картки хабів ChargeUA -->
</div>
```

Запитання 5. Яке призначення брейкпоїнтів sm:, md:, lg:, xl: та 2xl:?

🎯 **1. Коротка відповідь (для усного захисту):** Це контрольні точки (breakpoints), які встановлюють мінімальну ширину в'юпорту (@media (min-width: ...)), починаючи з якої набувають чинності відповідні утилітарні класи.

📚 **2. Поглиблене академічне обґрунтування:** Стандартна шкала контрольних точок Tailwind:
• sm: 640px — великі смартфони та ландшафтна орієнтація;
• md: 768px — планшети (iPad, тач-пристрої);
• lg: 1024px — невеликі ноутбуки та комп'ютери;
• xl: 1280px — повнорозмірні монітори та десктопи;
• 2xl: 1536px — широкі монітори Ultra-HD / 2K / 4K.
Завдяки Mobile First класу sm:text-base md:text-lg lg:text-xl розмір шрифту послідовно збільшується зі зростанням діагоналі дисплея.

💻 3. Практична реалізація та приклад у проєкті ChargeUA:

```
/* Еквівалентний медіа-запит, який генерує md:hidden */
@media (min-width: 768px) {
  .md\:hidden {
    display: none;
  }
}
```

Запитання 6. Для чого використовуються модифікатори станів hover:, focus:, active: та focus-visible:?

🎯 **1. Коротка відповідь (для усного захисту):** Вони дозволяють стилізувати інтерактивні псевдокласи браузера (:hover, :focus, :active, :focus-visible) безпосередньо всередині атрибута class елемента.

🔬 **2. Поглиблене академічне обґрунтування:** • hover: спрацьовує при наведенні курсора миші (наприклад, hover:bg-blue-700);
• active: активується в момент кліку або фізичного натискання сенсорного екрана (active:scale-95 створює тактильний ефект вдавлювання кнопки);
• focus: спрацьовує, коли елемент отримує фокус введення;
• focus-visible: сучасний стандарт доступності, що вмикає візуальне обведення (ring) ТІЛЬКИ при клавіатурній навігації (Tab) і не дратує користувачів мишкою.

💻 **3. Практична реалізація та приклад у проєкті ChargeUA:**

```
<button class="bg-blue-600 hover:bg-blue-700 active:scale-95 focus-visible:ring-4 focus-visible:ring-blue-300 transition-all">
  Зарядити зараз
</button>
```

Запитання 7. Які можливості надає розширення Tailwind CSS IntelliSense для VS Code?

🎯 **1. Коротка відповідь (для усного захисту):** Автодоповнення назв утилітарних класів, підказка точних числових CSS-значень при наведенні курсора, попередній перегляд кольорів безпосередньо в редакторі та валідація синтаксису.

🔬 **2. Поглиблене академічне обґрунтування:** Розширення аналізує конфігураційний файл tailwind.config та HTML-розмітку. Під час набору тексту воно виводить випадаючий список релевантних класів. При наведенні миші на клас (наприклад, p-6) спливає підказка: padding: 1.5rem; /* 24px */. Якщо клас містить колір (наприклад, bg-blue-600), ліворуч у рядку відображається кольоровий квадратик. Також плагін підкреслює помилки та конфліктні класи (Class Sorting & Linting).

💻 **3. Практична реалізація та приклад у проєкті ChargeUA:**

```
VS Code Extension ID: bradlc.vscode-tailwindcss
Підказка при наведенні на py-3: padding-top: 0.75rem; padding-bottom: 0.75rem;
```

Запитання 8. Чому згенерований штучним інтелектом код потребує рефакторингу (Refinement Workflow)?

🎯 **1. Коротка відповідь (для усного захисту):** ШІ-моделі часто допускають «галюцинації», використовують захардкоджені довільні значення (arbitrary values), ігнорують специфічні правила доступності (ARIA) або додають зайві незапитані елементи.

🔬 **2. Поглиблене академічне обґрунтування:** Під час генерації модель оптимізує результат під загальний візуальний вигляд, але часто не знає контексту повної дизайн-системи. Як результат:
1) Використання неекономних довільних класів типу bg-[#005EC9] замість семантичних токенів теми;
2) Відсутність обов'язкових атрибутів aria-expanded, aria-controls для інтерактивних скриптів;
3) Відсутність розмірів touch-target для мобільних екранів (кнопки менше 44px);
4) Надлишкові обгортки div. Студент як інженер зобов'язаний критично верифікувати код перед інтеграцією.

💻 **3. Практична реалізація та приклад у проєкті ChargeUA:**

```
<!-- До рефакторингу (код ШІ) -->
```

```
<div class="h-[38px] bg-[#005EC9]">Кнопка</div>
<!-- Після рефакторингу студентом -->
<button class="min-h-[44px] bg-blue-600 focus-visible:ring-4">Кнопка</button>
```

Запитання 9. Які вимоги до доступності вебінтерфейсу (Accessibility) перевірено під час виконання роботи?

🎯 **1. Коротка відповідь (для усного захисту):** Дотримання стандарту WCAG 2.2 рівня AA: контрастність тексту, мінімальний розмір сенсорних цілей 44x44px, доступність із клавіатури, наявність skip-link та ARIA-атрибути.

🔬 **2. Поглиблене академічне обґрунтування:** У проєкті ChargeUA перевірено:
1) Контраст кольорів: відношення яскравості білого тексту до фону #005EC9 становить понад 4.6:1 (успішно для нормального тексту);
2) Сенсорні зони (Touch Target Minimum WCAG 2.5.8): всі интерактивні елементи мають клас min-h-[44px];
3) Клавіатурна навігація: підтримка фокусу через focus-visible та закриття меню клавішею Escape;
4) Скрінрідери: наявність описових alt="..." для зображень, aria-label для кнопок-іконок та sr-only для заголовків.

💻 **3. Практична реалізація та приклад у проєкті ChargeUA:**
```
<!-- Посилання для швидкого переходу -->
<a href="#main-content" class="sr-only focus:not-sr-only focus:absolute z-50 bg-blue-600 text-white p-3">Перейти до вмісту</a>
```

Запитання 10. Для чого використовується режим Device Toolbar в Chrome DevTools?

🎯 **1. Коротка відповідь (для усного захисту):** Для повноцінної емуляції мобільних смартфонів і планшетів: тестування брейкпоїнтів, сенсорного введення (touch events), орієнтації екрана та обмеження швидкості мережі (Throttling).

🔬 **2. Поглиблене академічне обґрунтування:** Гарячі клавіші Ctrl+Shift+M активують вікно адаптивного тестування. Розробник може вибрати популярні профілі пристроїв (iPhone SE, iPhone 14 Pro, iPad Mini, Samsung Galaxy) або вручну тягнути за край вікна, відстежуючи точки перелому медіа-запитів. Режим дозволяє наочно побачити перемикання класів hidden / md:flex, перевірити відсутність горизонтального скролу та протестувати роботу висувного мобільного меню.

💻 **3. Практична реалізація та приклад у проєкті ChargeUA:**
```
Комбінація клавіш: F12 -> Ctrl + Shift + M -> вибір iPhone SE (375x812) -> тест меню.
```

Запитання 11. Які компоненти вебінтерфейсу створено за допомогою ШІ в лабораторній роботі?

🎯 **1. Коротка відповідь (для усного захисту):** Створено 5 ключових компонентів: 1) Primary CTA Button, 2) Адаптивний Header з мобільним Drawer, 3) Station Card (картка станції), 4) Адаптивна сітка Grid, 5) 4-колонковий Footer.

🔬 **2. Поглиблене академічне обґрунтування:** Кожен компонент пройшов шлях від текстового опису в інженерному промпті до автономного тестового файлу в каталозі components/ та подальшої збірки в головному

файлі index.html:
• Кнопка отримала мікроанімації та гарантовану доступність 44px;
• Шапка реалізує перетворення списку посилань на мобільний бургер;
• Картка містить динамічні статусні бейджі та цінові тарифи;
• Сітка автоматично балансує картки між 1, 2 та 3 колонками;
• Футер містить форму розсилки та живий статус доступності хабів мережі.

💻 3. Практична реалізація та приклад у проєкті ChargeUA:
`components/button.html | header.html | card.html | grid.html | footer.html`

Запитання 12. Що таке Preflight в Tailwind CSS і яке його призначення?

🎯 1. Коротка відповідь (для усного захисту): Preflight — це вбудований у Tailwind базовий набір нормалізуючих CSS-стилів, побудований на базі modern-normalize, який приводить стандартні стилі браузерів до єдиного нейтрального вигляду.

📚 2. Поглиблене академічне обґрунтування: Браузери за замовчуванням мають різні відступи та стилі елементів (User Agent Stylesheets). Preflight виконує:
1) Скидання зовнішніх відступів: margin: 0 для всіх тегів (body, h1-h6, p, ul);
2) Заголовки h1-h6 позбавляються вбудованого збільшеного розміру та жирності (вони стають font-size: inherit; font-weight: inherit);
3) Списки ul, ol позбавляються маркерів та відступів;
4) Зображення стають блочними елементами з display: block та max-width: 100% для запобігання виходу за межі батьківського контейнера;
5) Рамки за замовчуванням мають border: 0 solid #e5e7eb.

💻 3. Практична реалізація та приклад у проєкті ChargeUA:
```
/* Preflight автоматично запобігає вильоту зображень */
img, svg, video, canvas {
  display: block;
  max-width: 100%;
  height: auto;
}
```

Запитання 13. Які переваги має використання переходів (transition) та мікроанімацій (hover, active, focus) у вебінтерфейсі?

🎯 1. Коротка відповідь (для усного захисту): Плавні переходи надають користувачу миттєвий тактильний зворотний зв'язок (Micro-Interactions), підвищують сприйняття якості інтерфейсу, покращують UX та орієнтацію на сторінці.

📚 2. Поглиблене академічне обґрунтування: Без використання переходів зміна кольору чи положення елемента відбувається миттєво і виглядає різко. Класи transition-all duration-200 ease-in-out вмикають апаратне прискорення браузера (GPU Rendering). Коли користувач наводить курсор на кнопку ChargeUA, фон м'яко темнішає, тінь глибшає, а стрілочка робить плавний зсув праворуч. При натисканні (active:scale-95) кнопка пружно стискається, підтверджуючи успішність кліку перед відкриттям форми.

💻 3. Практична реалізація та приклад у проєкті ChargeUA:
`class="transition-all duration-200 hover:shadow-lg active:scale-95 ease-in-out"`

РОЗДІЛ 4. ВИСНОВКИ

1. У ході виконання Лабораторної роботи №6 успішно опановано парадигму компонентної верстки та утилітарного підходу Utility-First за допомогою фреймворку Tailwind CSS на прикладі вебплатформи мережі швидкісних зарядних хабів «ChargeUA».

2. Реалізовано повний інженерний цикл Prompt-to-UI взаємодії з ШІ-моделями (Claude 3.5 Sonnet, Google Gemini, Sequential Thinking MCP): здійснено проєктування промптів, генерацію автономних UI-компонентів, критичний аналіз коду та обов'язкове рафінування (Refinement Workflow).

3. Впроваджено мобільно-орієнтований підхід (Mobile-First): базова верстка проєкту оптимізована для смартфонів (375px), а утилітарні префікси sm:, md:, lg: та xl: плавно перебудовують інтерфейс для планшетів та десктопів без виникнення горизонтального скролу.

4. Синхронізовано дизайн-систему проєкту, розроблену в Figma у ЛР №4, із конфігурацією Tailwind через об'єкт tailwind.config, що забезпечило централізоване керування палітрою бренду (синій #005EC9, акцентний бірюзовий #00B4D8, нейтральний темний #0F172A).

5. Забезпечено найвищий рівень доступності за міжнародним стандартом WCAG 2.2 рівня AA (клас min-h-[44px] для сенсорного введення, skip-link, чіткий контур focus-visible:ring-4, закриття мобільного меню клавішею Escape), а також проведено перевірку сумісності у DevTools та W3C.